Lambda in Python

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

What is Lambda in Python?

A **lambda** is a **tiny function** useful when you want to define a function with **quick and simple logic**. Often, the lambda function is defined as the **argument** to other functions.

Syntax:

```
lambda arguments: expression
```

How does it work?

A lambda:

- can have **any number of arguments**, <u>but</u>
- must have **only one expression**, which gets returned automatically.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Typing the lambda

What is Callable?

- `Callable` is a type hint from the *typing* module used to specify that a variable is a function.

General form:

```
from typing import Callable
Callable[[ArgType1, ArgType2, ...], ReturnType]
```

Example:

```
from typing import Callable
# Function that takes two ints and returns an int
multiply:Callable[[int, int], int] = lambda a, b: a * b
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Lambda – Example/1

Basic lambda:

```
from typing import Callable
square:Callable[[int], int] = lambda x: x ** 2
print(square(5)) # Output: 25
```

This is functionally the same as:

```
def square(x:int) -> int:
    return x ** 2
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Lambda – Example/2

Basic lambda:

```
from typing import Callable
multiply:Callable[[float, float], float] = lambda a, b: a * b
print(multiply(3, 4)) # Output: 12
```

This is functionally the same as:

```
def multiply(a:float, b:float) -> float:
    return  a * b
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Lambda – Example/3

Basic lambda:

```
from typing import Callable
positive_or_negative:Callable[[int], str] = lambda x: "Positivo" if x > 0 else \
   "Zero o Negativo"
print(positive_or_negative(5)) # Output: Positivo
print(positive_or_negative(-3)) # Output: Zero o Negativo
```

This is functionally the same as:

```
def positive_or_negative(x:int) -> str:
    if x > 0:
       return "Positivo"
    else:
       return "Zero o Negativo"
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Lambda – Example/4

Using with filter():

```
nums:list[int] = [1, 2, 3, 4, 5]
evens:list[int] = list(filter(lambda x: x % 2 == 0, nums))
print(evens) # Output: [2, 4]
```

Using with sorted():

```
names:list[str] = ['Alice', 'Bob', 'Charlie']
sorted_by_length:list[str] = sorted(names, key=lambda name: len(name))
print(sorted_by_length) # Output: ['Bob', 'Alice', 'Charlie']
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

When to use and not use lambda?

📌 **When to *use* lambda**

- You need a function for a **short period of time**
- You don’t want to define a full def function
- You're passing a function as an argument (e.g., filter, sorted)

❌ **When *not* to use lambda**

- When the function is **complex or multi-line**
- When it needs to be **reused in many places** (prefer a def)
- When it hurts readability

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Filtering strings using regex + lambda

```
import re

words:list[str] = ["abc123", "456", "43", "hello", "98abc", "test999"]

# Keep only strings that contain only digits
only_digits:list[str] = list(filter(lambda x: re.fullmatch(r"\d+", x), words))
print(only_digits) # Output: ['456', '43']
```

Here:

`re.fullmatch(r"\d+", x)`: checks if the entire string *x* is only digits.

`lambda`: defines this condition inline.

`filter()`: applies `lambda` to every string in the list.

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Filtering strings using regex + lambda

```
import re

text:str = "Price: 100 dollars, Tax: 20 dollars"

# Multiply all numbers by 2 using a lambda
new_text:str = re.sub(r"\d+", lambda m: str(int(m.group()) * 2), text)
print(new_text) # Output: "Price: 200 dollars, Tax: 40 dollars"
```

Here:

`re.sub()`: replaces every number.

`lambda`: **dynamically computes** the replacement.

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY